

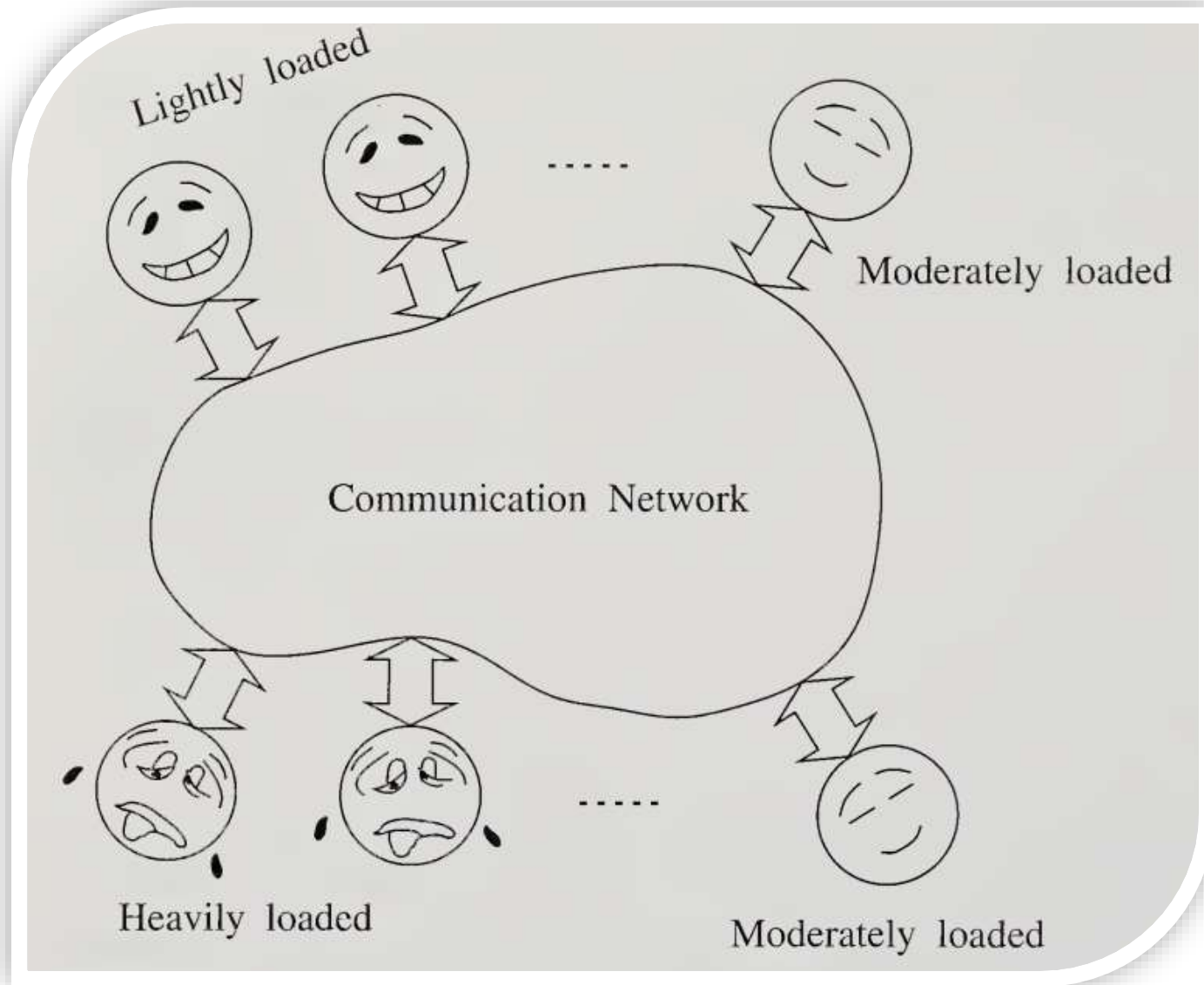
DISTRIBUTED SCHEDULING

- Distributed systems offer a tremendous processing capacity
- Good resource allocation schemes are needed
- **distributed scheduler**: a resource management component focusing on redistributing the load of the system among the computers to maximize the overall performance
- WANs have high communication delays
- Distributed scheduling is more suitable for DSs based on LANs

Need for load distributing

- **A locally distributed system:** a collection of autonomous computers, connected by a LAN
- Users submit tasks at their host computers for processing
- Need for load distributing rises because:
 - Random arrival of tasks and their random CPU service time
 - A distributed system may have a mix of heavily and idle or lightly loaded systems
- Hence, migrating a task to share or balance load can help

Need for load distributing



ISSUES IN LOAD DISTRIBUTING

- How to measure *Load*?
- Classification of Load Distributing Algorithms
- Load Balancing vs. Load Sharing
- Preemptive vs. Non-preemptive Transfers

How to measure Load?

- resource queue lengths & CPU queue length: good indicators of load
- they correlate well with the task response time
- Current CPU queue length as a load indicator can result in a node accepting tasks while other tasks it accepted earlier are still in transit
- When all the tasks that the node has accepted have arrived, the node can become overloaded and require further task transfers
- Increment CPU queue length whenever node accepts a remote task
- When task transfers fail, a timeout can be employed. After timeout, if the task has not yet arrived, CPU queue length is decremented

Classification of Load Distributing Algorithms

- **Dynamic load distributing algorithms** use system state information (the loads at nodes), at least in part, to make load distributing decisions
- **static algorithms** make no use of such information
- Here, decisions are **hard-wired in the algorithm using a priori knowledge of the system**
- **Adaptive algorithms: a special class of dynamic** load distributing algorithms that adapt their activities by dynamically changing the parameters of the algorithm to suit the changing system state.

Dynamic vs. Adaptive Algorithms

- E.g., a dynamic algorithm may continue to collect the system state irrespective of the system load.
- An adaptive algorithm, may discontinue the collection of the system state if the overall system load is high to avoid imposing additional overhead on the system.
- At such loads, all nodes are likely to be busy and attempts to find receivers are unlikely to be successful

Load Balancing vs. Load Sharing

- **unshared state:** in which one computer lies idle while at the same time tasks contend for service at another computer
- Both types of algorithms strive to **reduce the likelihood of an unshared state by transferring tasks to lightly loaded nodes**
- Load balancing algorithms **go a step further by attempting to equalize loads** at all computers
- A load balancing algorithm **transfers tasks at a higher rate than a load sharing algorithm**

Load Balancing vs. Load Sharing

- Task transfers are not instantaneous because of communication delays & delays during collection of task state.
- Delays in transferring a task increase the duration of an unshared state
- To avoid lengthy unshared states, anticipatory task transfers from overloaded computers to computers that are likely to become idle shortly can be used
- Anticipatory transfers increase the task transfer rate of a load sharing algorithm, making it less distinguishable from load balancing algorithms.
- load balancing: a special case of load sharing, performing a particular level of anticipatory task transfers

Preemptive vs. Non-preemptive Transfers

- **Preemptive task transfers:**

- ✓ involve the transfer of a task that is partially executed
- ✓ an expensive operation as the collection of a task's state can be difficult
- ✓ a task's state consists of a virtual memory image, a process control block, unread I/O buffers and messages, file pointers, timers that have been set, etc.

- **Non-preemptive task transfers:**

- ✓ involve transfer of tasks that have not begun execution and hence do not require the transfer of the task's state.

Components of a Load Distributing Algorithm

- **Transfer Policy:** determines whether a node is in a suitable state to participate in a task transfer
- **Selection policy:** determines which task should be transferred
- **Location policy:** determines to which node a task selected for transfer should be sent
- **Information policy:** responsible for triggering the collection of system state information
 - *A transfer policy typically **requires information on the local node's state** to make decisions*
 - *A location policy, is likely to **require information on the states of remote nodes** to make decisions*

Transfer Policy (TP)

- A large number of proposed TPs are threshold policies
- Thresholds are expressed in units of load.
- When a new task originates at a node, and the load at that node exceeds a threshold T , TP decides that node is a sender
- If the load at a node falls below T , TP decides that the node can be a receiver for a remote task.

Transfer Policy

- **Alternatives:**

1. **relative transfer policies:** consider the load of a node in relation to loads at other system nodes.
 - ✓ E.g., it might **consider a node** to be a suitable receiver **if its load is lower than that of some other node by at least some fixed δ**
2. **a node might be considered** a receiver if its **load is among the lowest** in the system

Selection Policy (SP)

- Once TP decides the node is a sender, a SP selects a task for transfer.
- **Simplest approach:**
 - ✓ select newly originated tasks that have caused the node to become a sender by increasing the load beyond the threshold
 - ✓ relatively cheap to transfer, as transfer is non-preemptive.
- Also, a task can be selected if the **estimated average execution time is greater than some execution time threshold**

Several factors in selecting a task

1. The **overhead incurred by the transfer** should be **minimal**. For example, a small task carries less overhead
2. The selected **task should be long lived so that it is worthwhile to incur the transfer overhead**
3. The **number of location-dependent system calls** made by the selected task **should be minimal**. Location-dependent calls must be executed on the node where the task originated, because they use resources that only exist at that node.

Location policy (LP)

- Responsibility of a LP is to find suitable nodes (senders or receivers) to share load
- **Decentralized policy finds a suitable node through polling:**
 - ✓ A node polls another node to find out whether it is suitable for load sharing.
 - ✓ Nodes can be polled either serially or in parallel (e.g., multicast).
 - ✓ A node can be selected for polling either randomly, based on the information collected during the previous polls, or on a nearest-neighbor basis
 - ✓ An alternative to polling is to broadcast a query to find out if any node is available for load sharing.

Location policy (LP)

✓ Centralized policy:

- ✓ a node **contacts one specified node called coordinator** to locate a suitable node for load sharing
- ✓ Coordinator **collects information about the system** (which is the responsibility of the information policy), and **transfer policy uses this information to select receivers.**

Information policy (IP)

- responsible for deciding **when information about the states** of other nodes in the system should be collected, **where it should be collected from**, and **what information should be collected**
- Types of information policies:
 - I. **Demand Driven:**
 - II. **Periodic:**
 - III. **State-change-driven:**

Demand Driven Information policy

- A node **collects state of other nodes only when it becomes a sender or a receiver** making it a **suitable candidate to initiate** load sharing
- demand-driven information policy is inherently a dynamic policy
- Demand-driven policies can be of three types:
 - I. Sender-initiated policies:** senders look for receivers to transfer their load.
 - II. Receiver-initiated policies:** receivers solicit load from senders.
 - III. Symmetrically initiated policy:** is a **combination of both**, where load sharing actions are triggered by the demand for extra processing power or extra work

Periodic Information policy

- Nodes **exchange load information periodically**
- Based on the information collected, the transfer policy at a node may decide to transfer jobs
- **benefits due to load distributing are minimal at high system loads** because most of the nodes in the system are busy
- Nevertheless, **overheads due to periodic information collection** continue to increase the system load

State-change-driven Information policy

- nodes disseminate state information whenever their state changes by a certain degree
- A state-change-driven policy differs from a demand-driven policy in that it disseminates information about the state of a node, rather than collecting information about other nodes

Stability

Two views of stability:

- **The Queuing-Theoretic Perspective**
- **The Algorithmic Perspective**

The Queuing-Theoretic Perspective

- Long term arrival rate of work to a system is greater than the rate at which the system can perform work
- CPU queues grow without bound
- Such a system is termed unstable
- Occurs when a load distributing algorithm performing excessive message exchanges to collect state information

The Algorithmic Perspective

- If an algorithm can perform fruitless actions indefinitely with finite probability, the algorithm is said to be unstable
- The transfer of a task to a receiver may increase the receiver's queue length to the point of overload, necessitating the transfer of that task to yet another node
- This process may repeat indefinitely
- Task is moved from one node to another in search of a lightly loaded node without ever receiving service

Load Distributing Algorithms

- **Sender-initiated:**

- ✓ load distributing activity **initiated by an overloaded node (sender)** that attempts to send a task to an underloaded node (receiver)
- ✓ We will study 3 sender-initiated algorithms

- **Receiver-initiated:**

- ✓ load distributing activity is **initiated from an underloaded node (receiver)** that is trying to obtain a task from an overloaded node (sender)

- Symmetric: initiated by both senders and receivers. Has advantages and disadvantages of both the approaches
- Adaptive: sensitive to state of the system

Sender-initiated Algorithms

- **Transfer Policy:** All three algorithms use thresholds TP
 - **Sender** if a new task originating at the node makes the queue length exceed a threshold T
 - **Receiver** if accepting a task will not make queue length exceed T.
- **Selection Policy: Only newly arrived tasks**
- **Location Policy:**
 - **Random:** Use no remote state information.
 - ✓ Task transferred to a node selected at random.
 - ✓ No need for state collection
 - ✓ **useless task transfers** can occur when a task is transferred to a node that is already heavily loaded (its queue length is above threshold)

Sender-initiated Algorithms

• *Location Policy:*

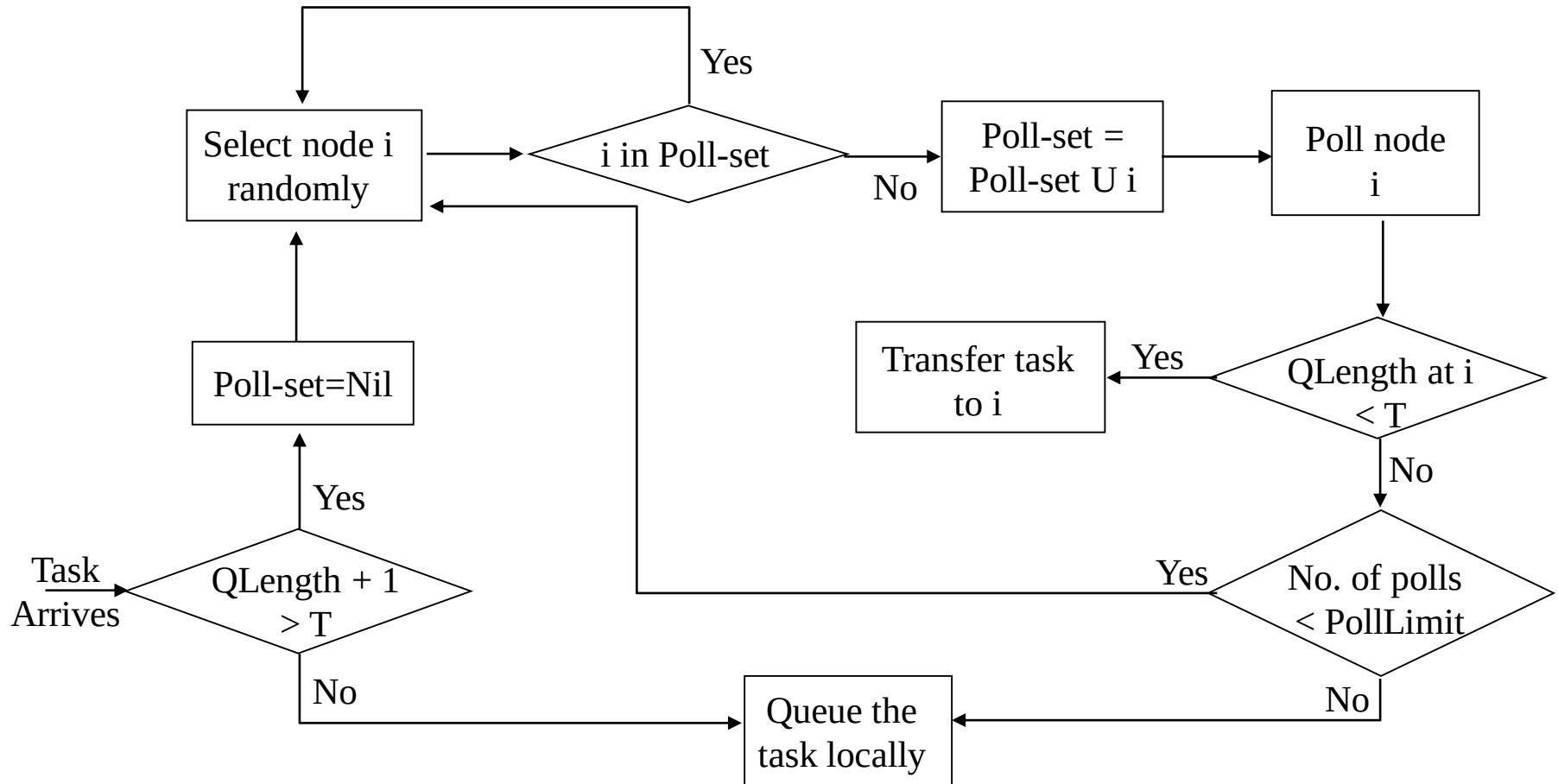
- **Threshold:** problem of useless task transfers can be avoided by polling a node (selected at random) to determine whether it is a receiver
- ✓ If so, task is transferred to the selected node, which must execute the task regardless of its state when the task actually arrives.
- ✓ Otherwise, another node is selected at random and polled.
- ✓ The number of polls is limited by a parameter called **PollLimit** to keep the overhead low.
- ✓ a sender node will not poll any node more than once
- ✓ If no suitable receiver node is found within the PollLimit polls, then the node at which the task originated must execute the task.

Sender-initiated Algorithms

• *Location Policy:*

- **Shortest:** two previous approaches make no effort to choose the best receiver for a task.
- ✓ a number of nodes (= PollLimit) are selected at random and are **polled to determine their queue length**
- ✓ **Node with shortest queue length is selected as the destination** for task transfer unless its queue length $> T$
- ✓ The destination node will execute the task regardless of its queue length at the time of arrival of the transferred task..

Sender-initiated load sharing with threshold location policy



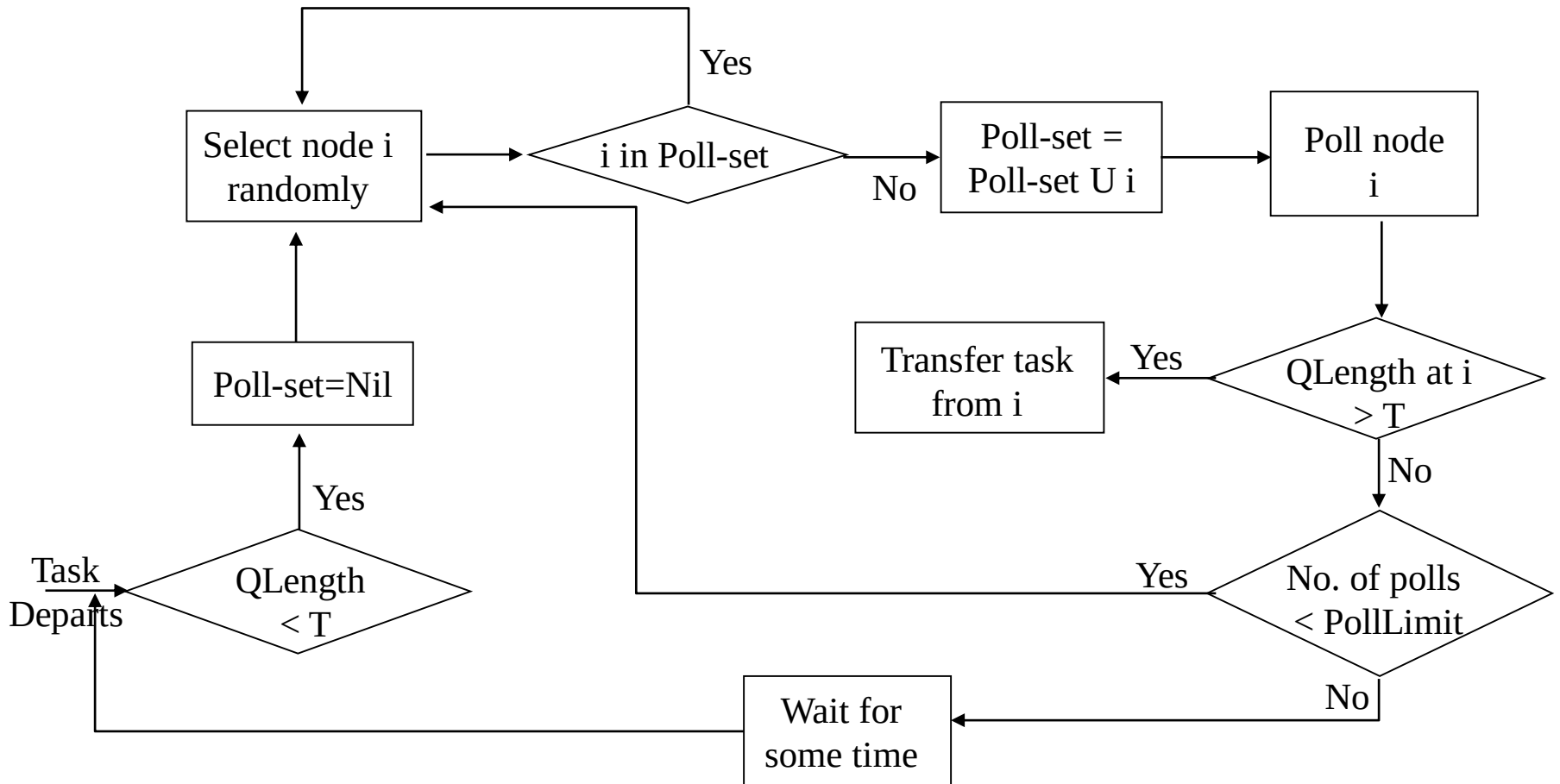
Cont...

- **Information Policy:** demand-driven.
- **Stability:** can become unstable at high loads.
 - At high loads, it may become difficult for senders to find receivers
 - Polling activity increases as the rate at which work arrives in the system increases, eventually reaching a point where the cost of load sharing is greater than the benefit
 - most of the available CPU cycles are wasted in unsuccessful polls and in responding to these polls
 - When the load due to work arriving and due to the load sharing activity exceeds the system's serving capacity, instability occurs.

Receiver-initiated Algorithms

- **Transfer Policy:** The transfer policy is triggered when a task departs. It uses thresholds. Queue lengths below T identifies receivers and those above T identifies senders.
- **Selection Policy:** as before.
- **Location Policy:** Polling.
 - A random node is polled to check if a task transfer would place its queue length below a threshold
 - If not, the polled node transfers a task.
 - Otherwise, poll another node till a static PollLimit is reached.
 - If all polls fail, wait until another task is completed before starting polling operation.
- **Information policy:** demand-driven.
- **Stability:** Not unstable since there are lightly loaded systems that have initiated the algorithm.

Receiver-initiated



Receiver-initiated

- Drawback:
 - Polling initiated by receiver implies that it is difficult to find senders with new tasks.
 - Reason: systems try to schedule tasks as and when they arrive.
 - Effect: receiver-initiated approach might result in preemptive transfers. Hence transfer costs are more.
 - Sender-initiated: transfer costs are low as new jobs are transferred and so no need for transferring task states.

Symmetric

- Senders search for receivers and vice-versa.
- Low loads: senders can find receivers easily. High loads: receivers can find senders easily.
- May have disadvantages of both: polling at high loads can make the system unstable. Receiver-initiated task transfers can be preemptive and so expensive.
- Simple algorithm: combine previous two approaches.
- Above-average algorithm:
 - Transfer Policy: Two adaptive thresholds instead of one. If a node's estimated average load is A , a higher threshold $TooHigh > A$ and a lower threshold $TooLow < A$ are used.
 - $Load < TooLow \rightarrow$ receiver. $Load > TooHigh \rightarrow$ sender.

Above-average Algorithm

- Location policy:
- Sender Component
 - Node with *TooHigh* load, broadcasts a *TooHigh* message, sets *TooHigh* timer, and listens for an *Accept* message.
 - A receiver that gets the (TooHigh) message sends an *Accept* message, increases its load, and sets *AwaitingTask* timer.
 - If the *AwaitingTask* timer expires, load is decremented.
 - On receiving the *Accept* message: if the node is still a sender, it chooses the best task to transfer and transfers it to the node.
 - When sender is waiting for *Accept*, it may receive a *TooLow* message (receiver initiated). Sender sends *TooHigh* to that receiver. Do step 2 & 3.
 - On expiration of *TooHigh* timer, if no *Accept* message is received, system is highly loaded. Sender broadcasts a *ChangeAverage* message.

Above-average Algorithm...

- Receiver Component
 - Node with *TooLow* load, broadcasts a *TooLow* message, sets a *TooLow* timer, and listens for *TooHigh* message.
 - If *TooHigh* message is received, do step 2 & 3 in Sender Component.
 - If *TooLow* timer expires before receiving any *TooHigh* message, receiver broadcasts a *ChangeAverage* message to decrease the load estimate at other nodes.
- Selection Policy: as discussed before.
- Information policy: demand driven. Average load is modified based on system load. High loads may have less number of senders progressively.
 - Average system load is determined individually. There is a range of acceptable load before trying to be a sender or a receiver.

Adaptive Algorithms

- Limit Sender's polling actions at high load to avoid instability.
- Utilize the collected state information during previous polling operations to classify nodes as: Sender/overloaded, receiver/underloaded, OK (in acceptable load range).
- Maintained as separate lists for each class.
- Initially, each node assumes that all others are receivers.
- Location policy at sender:
 - Sender polls the head of the receiver list.
 - Polled node puts the sender at the head of its sender list. It informs the sender whether it is a receiver, a sender, or a OK node.
 - If the polled node is still a receiver, the new task is transferred.
 - Else the sender updates the polled node's status, polls the next potential receiver.
 - If this polling process fails to identify a receiver, the task can still be transferred during a receiver-initiated dialogue.

Adaptive Algorithms...

- Location policy at receiver
 - Receivers obtain tasks from potential senders. Lists are scanned in the following order.
 - Head to tail in senders list (most up-to-date info used), tail to head in OK list (least up-to-date used), tail to head in receiver list.
 - Least up-to-date used in the hope that status might have changed.
 - Receiver polls the selected node. If the node is a sender, a task is transferred.
 - If the node is not a sender, both the polled node and receiver update each other's status.
 - Polling process stops if a sender is found or a static PollLimit is reached.

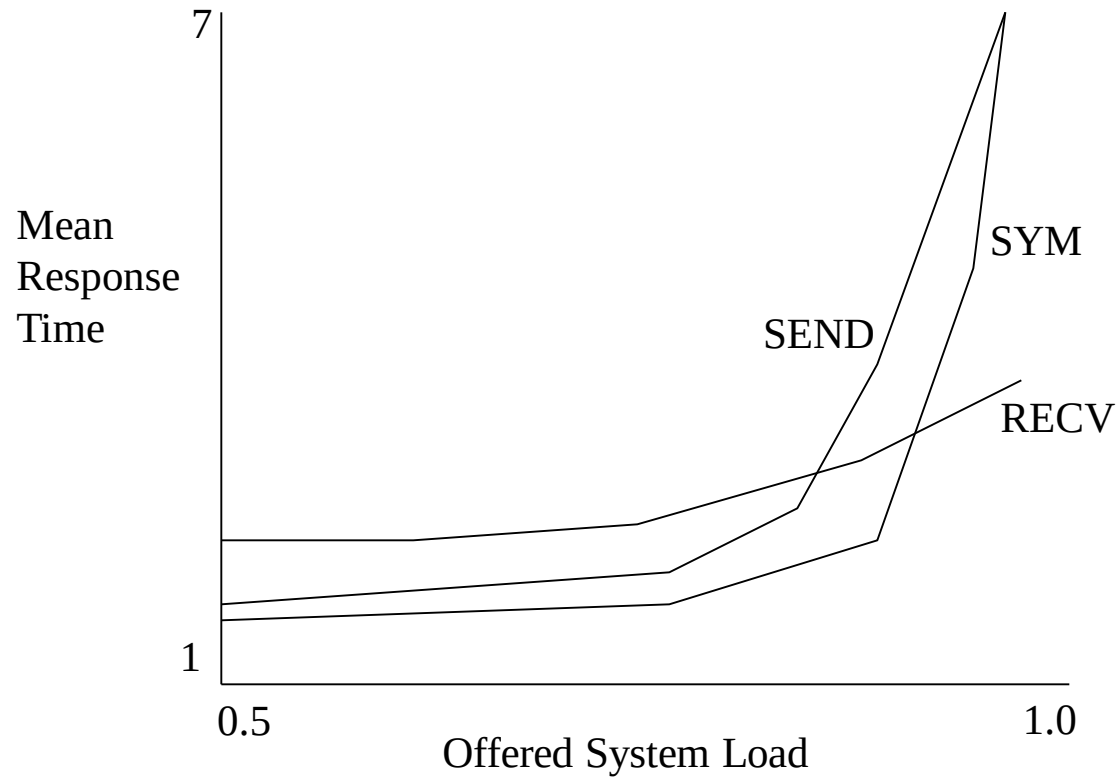
Adaptive Algorithms...

- At high loads, sender-initiated polling gradually reduces as nodes get removed from receiver list (and become senders).
 - Whereas at low loads, sender will generally find some receiver.
- At high loads, receiver-initiated works and can find a sender.
 - At low loads, receiver may not find senders, but that does not affect the performance.
- Algorithm dynamically becomes sender-initiated at low loads and receiver-initiated at high loads.
- Hence, algorithm is stable and can use non-preemptive transfers at low loads (sender initiated).

Selecting an Algorithm

- If a system never gets highly loaded, sender-initiated algorithms work better.
- Stable, receiver-initiated algorithms better for high loads.
- Widely fluctuating loads: stable, symmetric algorithms.
- Widely fluctuating loads + high migration cost for preemptive transfers: stable, sender-initiated algorithms.
- Heterogeneous work arrival: stable, adaptive algorithms.

Performance Comparison



Implementation Issues

- Task placement
 - A task that is yet to begin is transferred to a remote machine, and starts its execution there.
- Task migration
 - *State Transfer*:
 - State includes contents of registers, task stack, task status (running, blocked, etc.), file descriptors, virtual memory address space, temporary files, buffered messages.
 - Other info: current working directory, signal masks and handlers, references to children processes.
 - Task is frozen (suspended) before state transfer.
 - *Unfreeze*:
 - Task is installed at the new machine, unfrozen, and is put in the ready queue.

State Transfer

- Issues to be considered:
 - Cost to support remote execution including delays due to freezing the task.
 - Duration of freezing the task should be small. Otherwise it can result in timeouts of tasks interacting with the frozen one.
 - Transfer memory as they are referenced to reduce delay?
 - Residual Dependencies: amount of resources to be dedicated at the former host for the migrated task.
 - An implementation that does not transfer all the virtual memory address space at the time of migration, but transfers them as they are referenced.
 - Need for redirection of messages to the migrated task.
 - Location-dependent system calls at the former node.

State Transfer...

- Disadvantages of residual dependencies :
 - Affects reliability as it becomes dependent on former host
 - Affects performance since memory accesses can become slow as pages need to be transferred
 - Affects complexity as task states are distributed on several hosts.
- Location transparency:
 - Migration should hide the location of tasks
 - Message passing, process naming, file handling, and other activities should be transparent of actual location.
 - Task names and their locations can be maintained as hints. If the hints fail, they can be updated by a broadcast query or through a name server.